JAVA ARRAYS & ARRAYLIST

Complete Methods Reference • Concepts • Examples • Two Full-Length Programs

Is PDF mein kya milega?

• Array: declaration, initialization, access, traversal, copying, sorting, searching, comparison, filling, conversion, multidimensional arrays.

• java.util.Arrays class ke public method families aur unke important overloads.

• ArrayList: complete commonly used API — List, Collection, Iterator, ListIterator aur ArrayList-specific methods.

• Array aur ArrayList ke beech differences, common mistakes, time complexity aur best practices.

• End mein ek large integrated Array program aur ek large integrated ArrayList program.

Java version note: Examples modern Java syntax use karte hain. Core methods Java 8+ mein available hain; kuch newer methods (jaise mismatch, compare, sequenced collection methods) Java 9/21+ ke versions par depend karte hain. Apne JDK version ko check karna.

Learning approach: pehle concept samjho, phir method table dekho, phir code khud type karke run karo.

PART 1 — JAVA ARRAYS

Array kya hai? Array ek fixed-size data structure hai jo same data type ke multiple values ko ek hi variable name ke under store karta hai. Har element ka index 0 se start hota hai.

1. Declaration, creation aur initialization

```
int[] marks;                // declaration
marks = new int[5];         // creation: 5 integer slots
int[] scores = {80, 90, 75}; // declaration + initialization
int[] values = new int[]{2, 4, 6};
```

Array ki length create hone ke baad change nahi hoti. `marks.length` property hai — `marks.length()` method nahi.

2. Array access aur traversal

```
int[] a = {10, 20, 30};

System.out.println(a[0]); // 10
a[1] = 99;                // update

for (int i = 0; i < a.length; i++) {
    System.out.println(a[i]);
}

for (int value : a) {     // enhanced for-loop
    System.out.println(value);
}
```

3. Array ka apna member vs Arrays utility class

Expression / method	Meaning
arr.length	Array ki fixed length; field/property, method nahi.
arr[index]	Index se element read ya update.
Arrays.toString(arr)	1D array ko readable string mein convert.
Arrays.sort(arr)	Array ko sort karta hai.

Important: Java array object hai, lekin Array class jaisa `add()`, `remove()` ya `sort()` instance method nahi rakhta. Array operations ke liye `java.util.Arrays` utility class use hoti hai.

4. java.util.Arrays — public method families

Import: `import java.util.Arrays;` Neeche methods ko families mein diya gaya hai. Ek hi method ke multiple overloads primitive arrays, object arrays, ranges, comparators ya parallel processing ke liye ho sakte hain.

Method family	Kaam / important signatures
toString(...)	1D array ko string banana: toString(int[]), toString(Object[]), aur baaki primitive types.
deepToString(Object[])	Nested / multidimensional arrays ko readable string banana.
sort(...)	Ascending sort: sort(int[]), sort(int[], from, to), sort(T[]), sort(T[], Comparator), object-array range overloads.
parallelSort(...)	Parallel sorting; primitive arrays aur object arrays ke full-array / range overloads.
binarySearch(...)	Sorted array mein value search; primitive/object arrays, range aur Comparator overloads. Found index ya negative insertion-point
equals(...)	Do 1D arrays ke contents compare; primitive aur object array overloads.
deepEquals(Object[], Object[])	Nested arrays ke contents recursively compare.
hashCode(...)	Array contents ke basis par hash code; primitive/object array overloads.
deepHashCode(Object[])	Nested arrays ka recursive hash code.
fill(...)	Puri array ya [from,to) range ko given value se fill; primitive/object overloads.
setAll(...)	Generator function se array elements set: setAll(T[], IntFunction) / primitive generator versions.
parallelSetAll(...)	setAll jaisa, parallel execution ke saath.

Method family	Kaam / important signatures
copyOf(...)	Array ki copy, requested new length ke saath; primitive/object overloads.
copyOfRange(...)	[from,to) range ki copy; primitive/object overloads.
asList(T...)	Fixed-size List view banata hai; array-backed. Primitive array ko List<Integer> nahi banata.
stream(...)	Array/range se Stream; object aur primitive stream overloads.
spliterator(...)	Array/range ke liye Spliterator return karta hai.
compare(...)	Arrays ka lexicographical comparison; primitive/object arrays, range aur comparator overloads. Java 9+.
compareUnsigned(...)	Unsigned comparison for supported primitive integer arrays; Java 9+.
mismatch(...)	Pehla index return karta hai jahan arrays differ karte hain; range overloads bhi. Java 9+.
parallelPrefix(...)	Associative operation se cumulative/prefix result calculate karta hai; primitive/object overloads.
parallelSort / setAll / prefix	Parallel APIs; large arrays mein useful ho sakti hain, lekin har case mein faster hona guaranteed nahi.

Version note: `compare`, `compareUnsigned`, `mismatch` Java 9 se; `parallelPrefix` Java 8 se. `Arrays` class ke kuch overloads generic aur primitive-specific hote hain; exact signatures ke liye apne JDK ki Oracle API documentation dekho.

5. Arrays methods ke practical examples

```
import java.util.Arrays;

int[] nums = {40, 10, 30, 20};

System.out.println(Arrays.toString(nums)); // [40, 10, 30, 20]

Arrays.sort(nums);
System.out.println(Arrays.toString(nums)); // [10, 20, 30, 40]

int index = Arrays.binarySearch(nums, 30); // 2; array sorted hona chahiye

int[] copy = Arrays.copyOf(nums, 6);       // [10, 20, 30, 40, 0, 0]
int[] part = Arrays.copyOfRange(nums, 1, 3); // [20, 30]

Arrays.fill(copy, 100);                    // sab values 100

int[] x = {1, 2, 3};
int[] y = {1, 2, 3};
System.out.println(Arrays.equals(x, y));  // true

int[][] matrix = {{1, 2}, {3, 4}};
System.out.println(Arrays.deepToString(matrix)); // [[1, 2], [3, 4]]
```

6. Array sorting, searching aur comparison rules

• `Arrays.sort()` original array ko modify karta hai (in-place).

• `Arrays.binarySearch()` ko sorted array par use karo. Agar element nahi milta, result negative hota hai: `-(insertion point) - 1`.

• `Arrays.equals()` 1D contents compare karta hai; 2D arrays ke nested contents ke liye `deepEquals()` use karo.

• `copyOf()` shallow copy banata hai. Object array mein object references copy hote hain, objects khud clone nahi hote.

• `copyOfRange(a, from, to)` mein `from` included aur `to` excluded hota hai.

7. Multidimensional / jagged arrays

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

System.out.println(matrix[1][2]); // 6

for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

```
// Jagged array: rows ki lengths alag ho sakti hain
int[][] jagged = new int[3][];
jagged[0] = new int[]{1, 2};
jagged[1] = new int[]{3, 4, 5};
jagged[2] = new int[]{6};
```

8. Array time complexity — common operations

Operation	Typical time	Reason
Access by index	O(1)	Direct index access.
Update by index	O(1)	Direct index update.
Linear search	O(n)	Worst case mein sab elements check.
Binary search (sorted)	O(log n)	Search range har step mein half.
Sorting	Usually O(n log n)	Algorithm / data type par depend.
Copy / traverse	O(n)	Elements process/copy karne padte hain.

PART 2 — JAVA ARRAYLIST

ArrayList kya hai? `ArrayList` resizable-array implementation hai jo `java.util` package mein hoti hai. Iska size elements add/remove karne par badal sakta hai. Generic type se element type define karte hain.

```
import java.util.ArrayList;

ArrayList<Integer> numbers = new ArrayList<>();
ArrayList<String> names = new ArrayList<>();

numbers.add(10);
numbers.add(20);
System.out.println(numbers); // [10, 20]
```

Primitive types directly use nahi hote: `ArrayList` invalid hai. Wrapper classes use karo: `Integer`, `Double`, `Character`, `Boolean` etc.

1. ArrayList ke methods — complete practical API map

Method / family	Purpose
add(E e)	End mein element add.
add(int index, E element)	Specified index par insert; baaki elements shift.
addAll(Collection)	Collection ke sab elements end mein add.
addAll(int index, Collection)	Specified index se collection insert.
get(int index)	Index par element return.
set(int index, E element)	Existing element replace; old value return.
remove(int index)	Index wala element remove; removed element return.
remove(Object o)	Pehli matching value remove; success boolean.
removeAll(Collection)	Argument collection mein present elements remove.
retainAll(Collection)	Sirf argument collection mein present elements rakho.
removeIf(Predicate)	Condition true hone wale elements remove.
clear()	Saare elements remove.
size()	Current element count.
isEmpty()	List empty hai ya nahi.
contains(Object)	Element present hai ya nahi.
containsAll(Collection)	Kya saare given elements present hain.
indexOf(Object)	First matching index; absent ho to -1.
lastIndexOf(Object)	Last matching index; absent ho to -1.
iterator()	Forward traversal iterator.
listIterator() / listIterator(int)	Bidirectional traversal; optional starting index.
subList(int from, int to)	from included, to excluded range ka backed view.
toArray()	Object[] return.
toArray(T[] a)	Specified type ka array return.
toArray(IntFunction<T[]>)	Array generator ke saath typed array; Java 11+.
forEach(Consumer)	Har element par action perform.
replaceAll(UnaryOperator)	Har element ko operation ke result se replace.
sort(Comparator)	Comparator ke according list sort.
spliterator()	Spliterator return.
stream() / parallelStream()	Sequential / parallel Stream.
ensureCapacity(int)	Minimum internal capacity ensure; logical size nahi badalta.

Method / family	Purpose
trimToSize()	Internal capacity ko current size ke paas trim.
clone()	Shallow copy return; ArrayList implementation ka public method.
getFirst() / getLast()	First/last element; Java 21+.
removeFirst() / removeLast()	First/last element remove; Java 21+.
reversed()	Reverse-ordered view; Java 21+.
equals(Object) / hashCode()	List contents/order ke basis par equality/hash; inherited from AbstractList contract.
toString()	Readable list representation; inherited behavior.

Inherited API: ArrayList, `List`, `Collection`, `Iterable`, `AbstractList` aur `AbstractCollection` ke contracts/implementations se methods paati hai. Upar practical public API cover hai; `Object` class ke universal methods (getClass, wait, notify, etc.) ko collection operations nahi maana gaya.

2. add() ke overloads aur remove() ka important difference

```
ArrayList<Integer> a = new ArrayList<>();
a.add(10);       // end par add
a.add(1, 20);    // index 1 par add

a.remove(0);                 // index 0 remove
a.remove(Integer.valueOf(20)); // value 20 remove
```

`ArrayList` mein `remove(1)` index-based overload select karta hai. Value 1 remove karni ho to `remove(Integer.valueOf(1))` likho. Yeh common interview aur coding mistake hai.

3. Sorting, filtering, updating

```
ArrayList<Integer> nums = new ArrayList<>(Arrays.asList(5, 2, 8, 1));

nums.sort(null);                // natural ascending order
nums.replaceAll(n -> n * 2);    // every value double
nums.removeIf(n -> n > 10);     // values > 10 remove
nums.forEach(System.out::println);
```

Descending sort: `nums.sort(Collections.reverseOrder());` Iske liye `import java.util.Collections;` chahiye.

4. ArrayList traversal

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}

for (String item : names) {
    System.out.println(item);
}

list.forEach(item -> System.out.println(item));

var iterator = list.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

Iterator se traverse karte waqt safe removal ke liye `iterator.remove()` use karo. Enhanced for-loop ke andar directly list se remove karne par `ConcurrentModificationException` aa sakta hai.

5. ArrayList time complexity

Operation	Typical time	Notes
get(index) / set(index)	O(1)	Array-backed direct access.
add(element) at end	Amortized O(1)	Kabhi-kabhi resize/copy O(n).
add(index, element)	O(n)	Elements shift ho sakte hain.
remove(index)	O(n)	Elements shift ho sakte hain.
contains / indexOf	O(n)	Linear scan.

Operation	Typical time	Notes
sort	O(n log n)	Comparator / input ke hisaab se.
size / isEmpty	O(1)	Stored size field.

6. Array vs ArrayList

Feature	Array	ArrayList
Size	Fixed	Resizable
Can store	Primitive + objects	Objects; primitives ke liye wrappers
Access	arr[i]	list.get(i)
Length/count	arr.length	list.size()
Add/remove	Built-in add/remove nahi	add(), remove()
Utilities	java.util.Arrays	ArrayList / List / Collections
Performance	Low overhead, fixed size	Convenient dynamic storage; boxing possible

PART 3 — LARGE PROGRAM 1: ARRAY

Is program mein declaration, input, traversal, sum, average, min/max, search, sorting, reverse, copy aur frequency-style counting ka demonstration hai. Program ko `ArrayCompleteDemo.java` naam se save karke run karo.

```
import java.util.Arrays;
import java.util.Scanner;

public class ArrayCompleteDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("How many numbers? ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Size must be positive.");
            sc.close();
            return;
        }

        // 1. Create array and take input
        int[] numbers = new int[n];

        for (int i = 0; i < numbers.length; i++) {
            System.out.print("Enter number " + (i + 1) + ": ");
            numbers[i] = sc.nextInt();
        }

        // 2. Print complete array
        System.out.println("\nOriginal: " + Arrays.toString(numbers));

        // 3. Sum, minimum and maximum
        int sum = 0;
        int min = numbers[0];
        int max = numbers[0];

        for (int value : numbers) {
            sum += value;

            if (value < min) min = value;
            if (value > max) max = value;
        }

        double average = (double) sum / numbers.length;

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);
        System.out.println("Minimum = " + min);
        System.out.println("Maximum = " + max);

        // 4. Search for a value (linear search)
        System.out.print("\nEnter value to search: ");
        int target = sc.nextInt();

        int foundIndex = -1;
        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                foundIndex = i;
                break;
            }
        }

        if (foundIndex >= 0) {
            System.out.println(target + " found at index " + foundIndex);
        } else {
            System.out.println(target + " not found.");
        }

        // 5. Copy array
        int[] copied = Arrays.copyOf(numbers, numbers.length);
        System.out.println("Copied array: " + Arrays.toString(copied));

        // 6. Sort a separate copy (original remains unchanged)
        int[] sorted = Arrays.copyOf(numbers, numbers.length);
        Arrays.sort(sorted);
        System.out.println("Sorted array: " + Arrays.toString(sorted));

        // 7. Binary search must use sorted array
```

```
        System.out.print("Enter value for binary search: ");
        int binaryTarget = sc.nextInt();

        int binaryIndex = Arrays.binarySearch(sorted, binaryTarget);
        System.out.println("Binary search result: " + binaryIndex);
        // Negative result means value was not found.

        // 8. Reverse array into a new array
        int[] reversed = new int[numbers.length];

        for (int i = 0; i < numbers.length; i++) {
            reversed[i] = numbers[numbers.length - 1 - i];
        }
        System.out.println("Reversed: " + Arrays.toString(reversed));

        // 9. Fill a new array
        int[] filled = new int[5];
        Arrays.fill(filled, 7);
        System.out.println("Filled: " + Arrays.toString(filled));

        // 10. Compare arrays
        System.out.println("Original equals copy? "
                + Arrays.equals(numbers, copied));

        // 11. Count occurrences of each distinct value
        System.out.println("\nFrequency:");
        for (int i = 0; i < numbers.length; i++) {
            boolean alreadyCounted = false;

            for (int j = 0; j < i; j++) {
                if (numbers[i] == numbers[j]) {
                    alreadyCounted = true;
                    break;
                }
            }

            if (!alreadyCounted) {
                int count = 0;
                for (int value : numbers) {
                    if (value == numbers[i]) count++;
                }
                System.out.println(numbers[i] + " occurs " + count + " time(s)");
            }
        }

        sc.close();
    }
}
```

PART 4 — LARGE PROGRAM 2: ARRAYLIST

Is program mein add, insert, get, set, contains, indexOf, remove-by-index, remove-by-value, sort, reverse, replaceAll, removeIf, subList, iteration aur conversion demonstrate hote hain.

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Iterator;

public class ArrayListCompleteDemo {
    public static void main(String[] args) {

        // 1. Create ArrayList
        ArrayList<Integer> numbers = new ArrayList<>();

        // 2. Add elements
        numbers.add(40);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        System.out.println("After add: " + numbers);

        // 3. Insert at a particular index
        numbers.add(1, 15);
        System.out.println("After indexed add: " + numbers);

        // 4. Add all elements from another collection
        numbers.addAll(Arrays.asList(50, 60));
        System.out.println("After addAll: " + numbers);

        // 5. Read and update
        System.out.println("Element at index 0: " + numbers.get(0));
        numbers.set(0, 100);
        System.out.println("After set: " + numbers);

        // 6. Search
        System.out.println("Contains 30? " + numbers.contains(30));
        System.out.println("First index of 20: " + numbers.indexOf(20));
        System.out.println("Last index of 20: " + numbers.lastIndexOf(20));

        // 7. Size and empty check
        System.out.println("Size: " + numbers.size());
        System.out.println("Is empty? " + numbers.isEmpty());

        // 8. Remove by index and by value
        numbers.remove(0);                    // removes index 0
        numbers.remove(Integer.valueOf(60)); // removes value 60
        System.out.println("After remove: " + numbers);

        // 9. Sort ascending and descending
        numbers.sort(null);
        System.out.println("Ascending: " + numbers);

        numbers.sort(Collections.reverseOrder());
        System.out.println("Descending: " + numbers);

        // 10. Replace every value
        numbers.replaceAll(value -> value * 2);
        System.out.println("After replaceAll (*2): " + numbers);

        // 11. Remove elements that match a condition
        numbers.removeIf(value -> value > 80);
        System.out.println("After removeIf (>80): " + numbers);

        // 12. Traverse using for loop
        System.out.println("Using index loop:");
        for (int i = 0; i < numbers.size(); i++) {
            System.out.println(i + " -> " + numbers.get(i));
        }

        // 13. Enhanced for loop
        System.out.println("Using enhanced for:");
        for (int value : numbers) {
            System.out.println(value);
        }

        // 14. forEach
        System.out.println("Using forEach:");
```

```
        numbers.forEach(value -> System.out.println(value));

        // 15. Iterator traversal
        System.out.println("Using Iterator:");
        Iterator<Integer> iterator = numbers.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }

        // 16. Safe removal using Iterator
        Iterator<Integer> removeIterator = numbers.iterator();
        while (removeIterator.hasNext()) {
            if (removeIterator.next() == 20) {
                removeIterator.remove();
            }
        }
        System.out.println("After iterator removal: " + numbers);

        // 17. Sub-list (view): from inclusive, to exclusive
        if (numbers.size() >= 2) {
            System.out.println("SubList: " + numbers.subList(0, 2));
        }

        // 18. Convert ArrayList to array
        Integer[] array = numbers.toArray(new Integer[0]);
        System.out.println("Converted array: " + Arrays.toString(array));

        // 19. Clone (shallow copy)
        @SuppressWarnings("unchecked")
        ArrayList<Integer> cloned = (ArrayList<Integer>) numbers.clone();
        System.out.println("Cloned list: " + cloned);

        // 20. Capacity methods (internal storage only)
        numbers.ensureCapacity(100);
        numbers.trimToSize();

        // 21. Clear
        cloned.clear();
        System.out.println("Cloned list after clear: " + cloned);
        System.out.println("Is cloned list empty? " + cloned.isEmpty());

        // Original list is separate from the cloned list
        System.out.println("Original list remains: " + numbers);
    }
}
```

PART 5 — COMMON MISTAKES & PRACTICE

• Array ka `length` field hota hai; ArrayList ka `size()` method hota hai.

• `Arrays.sort(array)` array ko sort karta hai; `Collections.sort(list)` ya `list.sort(null)` list ko sort karta hai.

• `binarySearch()` se pehle array sort karo.

• `ArrayList` invalid hai; `ArrayList` use karo.

• `remove(2)` ArrayList mein index 2 remove karta hai; value 2 ke liye `remove(Integer.valueOf(2))`.

• `subList(from, to)` mein `to` index include nahi hota; result original list ka backed view hota hai.

• `Arrays.asList(array)` fixed-size list deta hai; add/remove karne par UnsupportedOperationException aa sakta hai.

• ArrayList ka `clone()` shallow copy hai. Nested mutable objects share ho sakte hain.

• ArrayList thread-safe nahi hai by default. Concurrent modification ke liye suitable concurrent collection ya synchronization choose karo.

Practice tasks

■ User se n numbers input lo; even aur odd numbers ke separate arrays banao.

■ Array mein second-largest distinct element find karo (sorting ke bina).

■ Array ko left rotate by k positions karo.

■ ArrayList se duplicate values remove karo, insertion order preserve karte hue.

■ ArrayList mein student names store karo; search, update, delete aur alphabetical sort implement karo.

■ 2D array ke har row ka sum aur har column ka sum calculate karo.

Official Java documentation

Arrays API: https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html

ArrayList API: https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/ArrayList.html

List API: https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/List.html

Collections API: https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Collections.html

Final learning instruction: Dono large programs ko sirf copy-paste mat karna. Har method ke saamne comment likho ki input kya hai, output kya hai, aur original data modify hota hai ya nahi.